SQL INJECTION - RETRIEVE HIDDEN DATA

TryHackMe Learning & Practical Report

Prepared by	Belal Rabea Khalifa
Program	IqraSec Academy, WAPT Internship (Cohort 02), Phase 1
Task	4. Common Web Vulnerabilities - SQL Injection
Platform	PortSwigger Web Security Academy
Lab	SQL injection vulnerability in WHERE clause allowing retrieval of hidden data (Apprentice)
Link	portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data
Focus	SQL injection, WHERE clause, category filter parameter
Status	Solved
Date	October 2026

1. Objective

Solve the PortSwigger lab "SQL injection vulnerability in WHERE clause allowing retrieval of hidden data": use the product category filter to make the application return hidden, unreleased products.

2. Concept / Understanding

- **SQL injection:** happens when user input is put directly into a database query without being treated as plain data, so the attacker's input can change the query's logic.
- In this lab, the category filter builds a query similar to `SELECT * FROM products WHERE category = 'Lifestyle' AND released = 1`. Only released products (released = 1) are normally shown.
- **Payload used:** `Lifestyle' OR 1=1--`. The single quote (') closes the category string early. `OR 1=1` is always true, so the WHERE condition becomes true for every row. `--` is an SQL comment, so the rest of the original query (AND released = 1) is ignored.
- Result: the query now matches every product, released or not, so hidden/unreleased products are returned too.

3. Practical Work

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data. I did the following steps:

#	Step	What I did / found
1	Opened the Lifestyle category and looked at the URL: /filter?category=Lifestyle.	Saw that the category value is sent directly as a URL parameter.
2	Replaced the category value in the URL with the payload: Lifestyle' OR 1=1--	URL became /filter?category=Lifestyle%27+OR+1=1--
3	Submitted the request (pressed Enter in the address bar).	The page loaded with extra products, including ones not shown before. The lab status changed to Solved.

4. Tools Used

- Web browser (address bar)
- PortSwigger Web Security Academy lab

5. Evidence

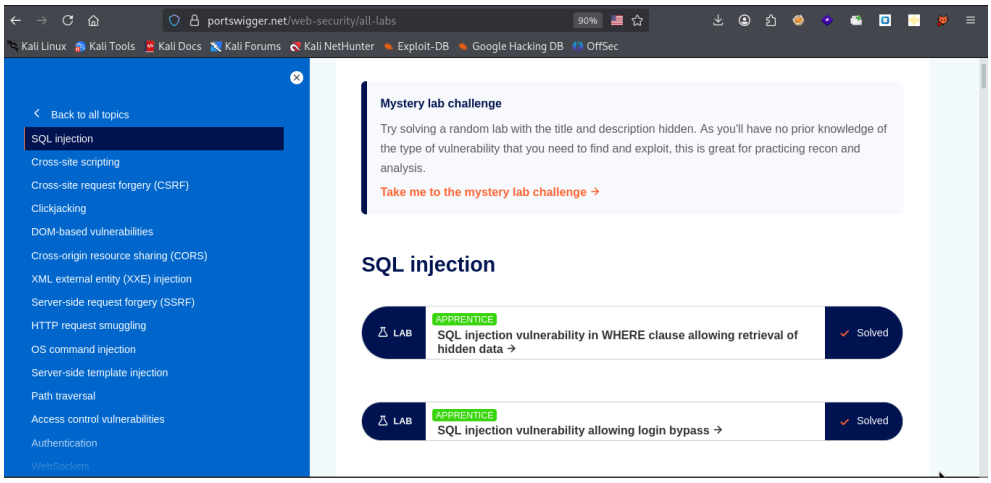


Figure 1: the SQL injection labs list. Lab "SQL injection vulnerability in WHERE clause allowing retrieval of hidden data", marked Solved.



Figure 2: the payload Lifestyle' OR 1=1-- typed into the category parameter in the URL bar, before submitting (status: Not solved).

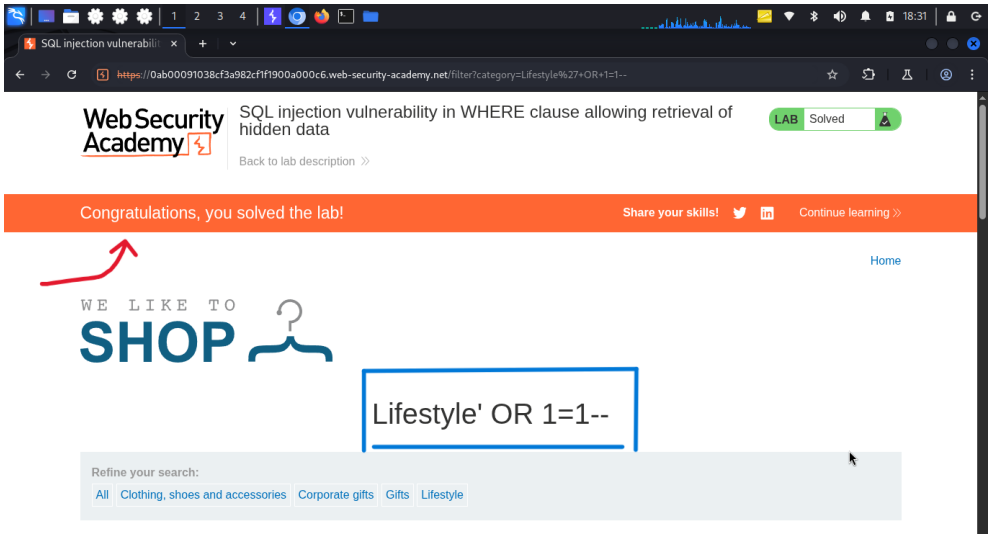


Figure 3: after submitting the payload, the page shows "Congratulations, you solved the lab!" and the full URL with the encoded payload.

6. Result

Solved the lab. The payload Lifestyle' OR 1=1-- made the WHERE clause always true, so the application returned all products, including the hidden unreleased ones.

7. Key Learnings

- Injecting OR 1=1 into a WHERE clause can override the intended filter and return all rows.
- The -- comment syntax removes the rest of the original query, so the injected condition is the only one left.
- A parameter that looks harmless (like a category filter) can still be a point of SQL injection if it is not handled safely.
- Testing with a single quote (') is a simple way to check if input reaches the SQL query directly.

8. Impact and Remediation

Impact: an attacker can bypass filters to read data that should be hidden (like unreleased products), and in other cases SQL injection can expose far more sensitive data such as other users' records or credentials.

Remediation:

- Use parameterized queries / prepared statements, so user input is always treated as data and never as part of the SQL command.
- Validate input against an allow-list of expected values (for example, only accept known category names).
- Do not build SQL queries by joining strings with user input.